

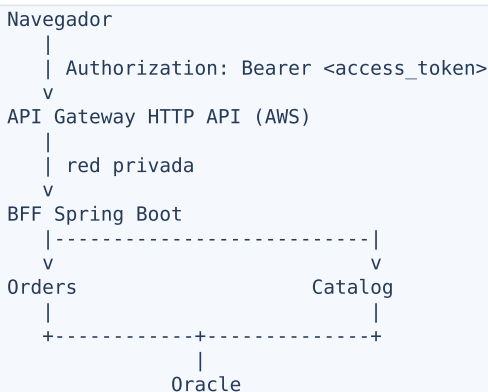
Manual técnico de Pedidos360

1. Propósito del proyecto

Pedidos360 es una aplicación web para consultar y administrar pedidos, productos, inventario y reservas de stock. El frontend es una SPA React + Vite + TypeScript con Microsoft Entra ID. El backend está compuesto por un BFF Spring Boot, dos microservicios Spring Boot y Oracle.

Este manual describe la organización del código, el flujo de autenticación, la ejecución local y las fronteras de seguridad. No contiene contraseñas, tokens, connection strings ni secretos de despliegue.

2. Arquitectura



En local, el navegador usa `http://localhost:8080` como URL del BFF. En AWS, el navegador debe usar exclusivamente el endpoint HTTPS de API Gateway. Oracle, orders, catalog y BFF permanecen en redes privadas.

3. Componentes

Componente	Ruta	Responsabilidad
Frontend	frontend-pedidos360/	SPA React, rutas, guardas, MSAL, cliente HTTP y vistas
BFF	ms-pedidos360-bff/	Segunda validación JWT, roles, CORS y proxy privado
Orders	ms-pedidos360-orders/	Pedidos, estados, propiedad del cliente y orquestación de stock
Catalog	ms-pedidos360-catalog/	Productos, stock y reservas internas
Infraestructura	infra/, docker-compose.yml	Integración local, API Gateway, OpenAPI y Terraform
Base local	Oracle 23 Free	Persistencia separada de catálogo y pedidos
Utilidades	scripts/	Arranque, detenimiento, smoke test y provisioning local

4. Flujo de una solicitud

1. React inicia sesión con Authorization Code + PKCE mediante `@azure/msal-react` y `@azure/msal-browser`.
2. MSAL obtiene un access token para el scope de la API.
3. El cliente HTTP de React agrega `Authorization: Bearer` a las rutas `/api/`.
4. API Gateway valida el token en el despliegue AWS.
5. El BFF vuelve a validar firma, issuer, audience y vigencia.
6. El BFF reenvía la solicitud a orders o catalog usando la red privada.
7. El microservicio aplica autorización por rol y ejecuta la operación.
8. La respuesta regresa al frontend con el mismo contrato HTTP.

El BFF no tiene JDBC, JPA, datasource ni credenciales de Oracle. Los controles de rol del frontend son solo de usabilidad; la autorización real permanece en el BFF y los microservicios.

5. Ejecutar el proyecto localmente

Requisitos

- Docker Engine y Docker Compose v2.

- JDK 17 y Maven 3.9 o superior para los backends.
- Node.js 26 y npm para React/Vite.
- Al menos 8 GB de memoria disponible para el stack con Oracle.
- Microsoft Entra ID configurado para la SPA y la API.

Configuración

1. Copiar .env.example a .env.
2. Completar los identificadores de Entra y las contraseñas locales de Oracle.
3. No compartir ni versionar .env.
4. Ejecutar:

```
chmod +x scripts/run-local.sh scripts/stop-local.sh scripts/ensure-oracle-users.sh
./scripts/run-local.sh
```

El script valida la configuración, construye las imágenes y espera los healthchecks. La URL local del frontend es `http://localhost:4200` y el BFF está en `http://localhost:8080`.

Frontend sin Docker

```
npm --prefix frontend-pedidos360 ci
npm --prefix frontend-pedidos360 run dev
```

Vite escucha en `http://localhost:4200`. El cliente usa el BFF local y requiere un token de Entra válido para las operaciones protegidas.

Detener y conservar datos

```
./scripts/stop-local.sh
```

Detener y borrar el volumen

Esta operación elimina los datos locales de Oracle:

```
docker compose --env-file .env -f docker-compose.yml down --volumes
```

6. Frontend React

Estructura

- `src/main.tsx`: punto de entrada, MsalProvider, contexto de autenticación y router.
- `src/App.tsx`: rutas públicas, rutas protegidas y guards.
- `src/auth/msal.ts`: configuración de la SPA y authority /v2.0.
- `src/auth/AuthContext.tsx`: cuenta activa, claims, roles, oid, login y logout.
- `src/auth/RedirectHandler.tsx`: coordinación del callback de redirect después de la inicialización de MSAL.
- `src/api/client.ts`: cliente fetch, Bearer y normalización de errores.
- `src/api/services.ts`: contrato de pedidos y catálogo, normalización y filtros de cliente.
- `src/pages/`: vistas React de login, dashboard, pedidos y catálogo.
- `src/utils/claims.ts`: funciones puras para claims, roles y oid.
- `src/styles.css`: estilos globales y responsive.

Configuración de build

La configuración se encuentra en `src/config/environment.ts`. Para un ambiente, el script genera ese archivo durante el build con:

- `ENTRA_TENANT_ID`.
- `ENTRA_CLIENT_ID`.
- `API_SCOPE`.
- `API_BASE_URL`.
- `REDIRECT_URI`.
- `POST_LOGOUT_REDIRECT_URI`.

La imagen Docker ejecuta `npm run generate:environment` antes de `npm run build`. Los valores deben estar disponibles durante el build; asignar una variable al contenedor en tiempo de ejecución no reescribe un bundle estático.

MSAL y seguridad

- La SPA es pública y usa Authorization Code + PKCE.
- `redirectUri` y `postLogoutRedirectUri` deben coincidir exactamente con Entra.
- El scope local es `api://150f51db-4084-4979-b1a1-e6a6e7893a01/access_as_user`.
- `MsalProvider` inicializa MSAL y procesa el callback una sola vez por instancia.
- `AuthContext` adquiere el token silenciosamente y combina claims del identificador y del access token.
- `oid` usa fallback a sub cuando no existe `oid`.
- La decodificación visual de un JWT no valida criptográficamente el token. La validación real se hace en API Gateway, BFF y microservicios.
- Los servicios locales aceptan solo el issuer y la audiencia exactos del tenant en formato v2 o el formato v1 equivalente de Entra (`sts.windows.net` y `api://{client-id}`); no se validan comodines.
- El código no contiene ni muestra contraseñas, client secrets ni tokens; MSAL administra únicamente la caché temporal de la sesión.

Rutas principales

- `/login`: inicio de sesión público.
- `/dashboard`: resumen y pedidos recientes; requiere sesión.
- `/orders`: lista, creación y seguimiento; requiere sesión.
- `/catalog`: administración de productos y stock; requiere Admin u Operador.

`RequireAuth` protege las vistas privadas. `RequireCatalogRole` espera la inicialización de MSAL y la lectura de claims antes de autorizar o redirigir.

Cliente HTTP y contrato

React consume solamente estas rutas públicas del BFF:

- GET/POST `/api/orders`.
- GET/PUT/DELETE `/api/orders/{id}`.
- PATCH `/api/orders/{id}/status`.
- GET/POST `/api/catalog/products`.
- GET/PUT/DELETE `/api/catalog/products/{id}`.
- PATCH `/api/catalog/products/{id}/stock`.

El cliente agrega el Bearer y convierte errores HTTP en `ApiError`. Los identificadores se codifican con `encodeURIComponent`. No se consumen endpoints internos de reservations ni URLs privadas de microservicios.

Modificar una vista

1. Abrir `frontend-pedidos360/src/pages/`.
2. Modificar el componente React y su tipo de datos.
3. Mantener las rutas y el contrato definidos en `infra/aws/openapi.yaml`.
4. Añadir o actualizar pruebas en `src/**/*.test.ts` o `src/**/*.test.tsx`.
5. Ejecutar:

```
npm --prefix frontend-pedidos360 test
npm --prefix frontend-pedidos360 run build
```

7. BFF Spring Boot

Responsabilidades

`ms-pedidos360-bff` valida el JWT con Spring Security Resource Server. Configura CORS para el origen exacto del frontend y aplica reglas de autorización a las operaciones de administración.

El proxy utiliza `RestClient` y `JdkClientHttpRequestFactory` para soportar todos los métodos HTTP, incluido PATCH.

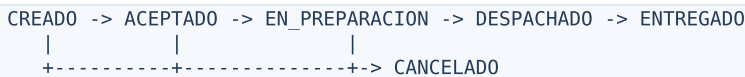
Archivos importantes

- `src/main/java/cl/pedidos360/bff/config/SecurityConfig.java`: JWT, CORS y reglas HTTP.
- `src/main/java/cl/pedidos360/bff/config/AudienceValidator.java`: audience obligatorio.
- `src/main/java/cl/pedidos360/bff/web/BffController.java`: contrato público del BFF.
- `src/main/java/cl/pedidos360/bff/gateway/BffProxyService.java`: comunicación privada con microservicios.
- `src/main/resources/application.yml`: variables de configuración.

El BFF nunca debe conectarse directamente a Oracle.

8. Orders

Orders es el dueño de los pedidos. Persiste en el esquema PEDIDOS360_ORDERS y aplica la máquina de estados:



Al aceptar un pedido se reserva stock. Al cancelar una reserva activa se libera. La reserva es idempotente usando orderId.

La interfaz React solo ofrece las transiciones definidas en `STATUS_TRANSITIONS`; el backend vuelve a validarlas.

9. Catalog

Catalog es el dueño de productos, stock y reservas. Persiste en PEDIDOS360_CATALOG.

Las operaciones de stock y el CRUD de productos requieren Admin u Operador. Los usuarios con rol Cliente pueden consultar productos activos para crear pedidos, pero no pueden ingresar a la ruta administrativa.

10. Oracle y Docker

docker-compose.yml define una red backend privada, una red edge para frontend/BFF y una red de salida para los servicios que necesitan Entra. Solo frontend y BFF publican puertos locales.

Hay dos usuarios Oracle independientes:

- PEDIDOS360_CATALOG.
- PEDIDOS360_ORDERS.

ensure-oracle-users.sh valida o crea ambos principales al iniciar Oracle. El script no imprime contraseñas.

11. AWS

Terraform crea el esqueleto de API Gateway, authorizer JWT, CORS, VPC Link e integración privada. No crea por sí solo la VPC, el runtime compute, el listener privado ni Oracle gestionado.

Antes de terraform apply deben existir:

- VPC y subredes privadas.
- Route tables y resolución de DNS.
- Security groups mínimos.
- ECS/Fargate o runtime equivalente.
- Oracle privado.
- Secrets Manager o mecanismo equivalente.
- Listener y health checks privados.

La URL pública que recibe React debe ser el output de API Gateway. Nunca se debe poner la URL privada de orders, catalog, BFF u Oracle en el frontend.

12. Pruebas

Backends

```
mvn -f ms-pedidos360-catalog/pom.xml clean verify
mvn -f ms-pedidos360-orders/pom.xml clean verify
mvn -f ms-pedidos360-bff/pom.xml clean verify
```

Frontend

```
npm --prefix frontend-pedidos360 ci
npm --prefix frontend-pedidos360 test
npm --prefix frontend-pedidos360 run build
npm --prefix frontend-pedidos360 audit --omit=dev --audit-level=high
```

Las pruebas frontend cubren helpers de claims, roles, transiciones, cliente HTTP, errores, servicios y login. También deben verificarse los flujos de formularios y autorización en la matriz de entrega.

Integración

```
./scripts/smoke-local.sh
```

El smoke test verifica autenticación, autorización, propiedad del pedido, transición inválida, reserva de stock, cancelación y errores de red.

13. Solución de problemas

Página en blanco

- Verificar docker compose ps.
- Abrir <http://localhost:4200/healthz>.
- Recargar con Ctrl+Shift+R.
- Confirmar que el navegador usa el bundle actualizado.

Error de redirect URI

La URI enviada por React debe coincidir exactamente con la registrada en Entra. Para local debe ser <http://localhost:4200/login>.

Respuesta 401

- No existe un Bearer válido.
- El token no corresponde al issuer o audience configurados.
- El token puede haber expirado.
- API_BASE_URL no coincide con la URL usada para configurar MSAL y el cliente.

Respuesta 403

El token es válido, pero el rol no permite la operación. Los roles esperados son Admin, Operador y Cliente.

Disco lleno

Docker y Oracle pueden ocupar la partición raíz. No borrar el volumen para recuperar espacio. Primero eliminar imágenes dangling no usadas y, si es necesario, mover el data-root de Docker a /home.

14. Reglas de seguridad

- No mostrar, registrar ni guardar tokens, contraseñas o secretos.
- No incluir .env en Git ni en el ZIP.
- Validar issuer, audience, firma y vigencia en cada frontera backend.
- Mantener catalog, orders, BFF y Oracle sin puertos públicos.
- Usar CORS con el origen exacto, nunca con * y credenciales.
- Aplicar mínimo privilegio a usuarios Oracle, Docker, AWS y Entra.
- Mantener las URL privadas fuera del bundle del frontend.

- Registrar decisiones de arquitectura y cambios de contrato.

15. Resumen de modificación rápida

```
| Necesidad | Archivo o servicio principal |
|---|---|
| Cambiar una pantalla | frontend-pedidos360/src/pages/ |
| Cambiar login o claims | frontend-pedidos360/src/auth/ |
| Cambiar el cliente HTTP | frontend-pedidos360/src/api/ |
| Cambiar una ruta API | infra/aws/openapi.yaml y BffController.java |
| Cambiar reglas de pedido | OrderService.java y OrderTransitionPolicy.java |
| Cambiar stock | StockReservationService.java |
| Cambiar tabla Orders | ms-pedidos360-orders/src/main/resources/db/migration/ |
| Cambiar tabla Catalog | ms-pedidos360-catalog/src/main/java/.../domain/ y configuración de esquema |
| Cambiar contenedores | docker-compose.yml y Dockerfiles |
| Cambiar infraestructura AWS | infra/aws/terraform/ y docs/DESPLIEGUE_AWS.md |
| Ejecutar todo local | ./scripts/run-local.sh |
| Detener sin borrar datos | ./scripts/stop-local.sh |
```